

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO2 ASSESSMENT TOOL 2 — Architecture Design Assignment

Course Code:	CSA1007	Course Title:	Software Engineering
CO Assessed:	CO2	Assessment:	Assessment Tool 2 – Architecture Design Assignment
Weightage:	20%	Total Marks:	25
CO2 Statement:	<i>Perform detailed requirements analysis, design scalable architectures, and prioritize features with a focus on cross-disciplinary skills.</i>		
Student Name:	N.MadhuSudhan	Reg. No.:	192425303
Date:	28/08/2026	Duration:	60 minutes

Code of Conduct

I, N.Madhusudhan (192425303), certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: N.MadhuSudhan

Annexure A — Architecture Design Assignment

Problem Statement: Smart Grid Cybersecurity Management Platform

1. Analyze System Requirements

1.1 System Overview and Objectives

The assigned problem statement calls for a Smart Grid Cybersecurity Management Platform that monitors grid devices, detects cyber attacks and intrusions, secures smart meters and substations, predicts threats using AI, and generates consolidated security and compliance reports. Critical energy infrastructure is characterized by heterogeneous legacy and IoT devices, strict availability requirements, and continuous high-frequency telemetry, so the architecture must support independent scaling of AI-driven detection, resilient ingestion from thousands of field devices, and consolidated reporting for regulators and executives.

The convergence of operational technology (SCADA, substation controllers) with information technology networks means that a security incident can propagate from a single compromised field device into a much wider operational disruption if it is not detected and contained quickly. The platform must therefore be designed so that detection, response, and reporting capabilities can each evolve and scale on their own timeline, without requiring the entire system to be redeployed whenever one capability changes.

The core system objectives that drive the architectural design are:

- Monitor smart grid devices, substations, and smart meters in real time.
- Detect cyber attacks, malware, and intrusions across IT and OT segments.

- Predict emerging threats using AI/ML models and generate risk scores.
- Coordinate automated and analyst-driven incident response.
- Track compliance against regulatory cybersecurity standards.
- Generate consolidated security performance reports for stakeholders.

1.2 Functional Requirements Influencing the Architecture

The following functional capabilities directly shape how the system must be partitioned into components and how those components must communicate:

- An AI/ML threat detection and prediction engine that runs independently of transactional workloads, so it can be scaled and retrained without affecting other services.
- A device and substation monitoring module that continuously tracks status and communication health of field assets.
- An intrusion detection module that correlates network flow data with device telemetry across IT and OT segments.
- A risk scoring module that quantifies severity and prioritizes detected events for response.
- An incident response module that executes or authorizes containment actions and tracks resolution.
- A compliance and reporting module that aggregates data from every other module for regulators and executives.
- A user and access management module enforcing role-based access across internal and supplier-facing users.

1.3 Non-Functional Requirements and Quality Attributes

Because threat detection, device monitoring, incident response, and reporting have very different load patterns, resource needs, and release cadences, the architecture must let each concern evolve, scale, and fail independently. The key quality attributes identified are summarized below.

Quality Attribute	Requirement Driving the Architecture
Scalability	The system must scale horizontally as device count and regions grow, and must allow the AI detection engine to scale independently of transactional modules during peak load.
Performance	Threat alerts should be generated within 2 seconds of detection; dashboards should load within 3 seconds; device status must reflect within 15 minutes of a telemetry update.
Reliability / Availability	The platform must maintain at least 99.9% uptime; a failure in one module (e.g., device monitoring) must not bring down threat detection or reporting.

Quality Attribute	Requirement Driving the Architecture
Security	Data must be encrypted in transit and at rest, with role-based access control and multi-factor authentication for administrative and field-facing roles.
Maintainability	Detection and prediction models must be retrainable and independently deployable without a full system redeployment.
Interoperability	The platform must integrate with existing SCADA/IED systems, smart meter gateways, and regulatory compliance reporting systems.

2. Select a Suitable Software Architecture

2.1 Selected Architecture: Cloud-Native Microservices with Event-Driven Communication

A Microservices Architecture, supported by an API Gateway for client-facing access and an event-driven message broker for asynchronous data flows, is selected as the most suitable architectural style for the Smart Grid Cybersecurity Management Platform.

2.2 Justification

- Independent, heterogeneous modules: AI-based threat detection is a compute-intensive workload, while device monitoring, incident response, and reporting are transactional and query-driven. Microservices allow each module to be built, scaled, and deployed on the stack best suited to its own workload.
- Independent scalability: Attack surges or seasonal load spikes can be absorbed by scaling only the affected service, rather than the entire monolith, directly satisfying the scalability requirement.
- Fault isolation: If the device monitoring service experiences a field-gateway outage, threat detection, risk scoring, and reporting continue to operate normally, supporting the 99.9% availability target.
- Asynchronous, high-volume data ingestion: Telemetry from substations and smart meters arrives continuously and at high frequency. An event-driven backbone decouples producers (field devices) from consumers (detection, monitoring), preventing slow consumers from blocking upstream systems.
- Independent release cycles: Detection models need frequent retraining and redeployment; microservices allow the detection service to be updated without redeploying incident response or reporting services, supporting maintainability.
- Straightforward integration: An API Gateway provides a single, secured entry point for field systems and internal clients, simplifying authentication, rate limiting, and monitoring of external integrations (SCADA, meters, regulators).

A pure monolithic or simple layered architecture was considered but rejected, since it would force the compute-intensive detection workload and high-frequency telemetry streams to share the same scaling and deployment unit as transactional modules, making it difficult to meet performance and availability

targets during attack surges. A Service-Oriented Architecture (SOA) with a heavyweight enterprise service bus was also considered, but was judged to introduce unnecessary complexity and a central point of failure compared to a lightweight, independently deployable microservices approach combined with a message broker.

3. Software Architecture Diagram

The diagram below illustrates the overall structure of the proposed platform, including the presentation layer, API Gateway, the microservices layer, the event-driven message broker, the polyglot data layer, and external system integrations.

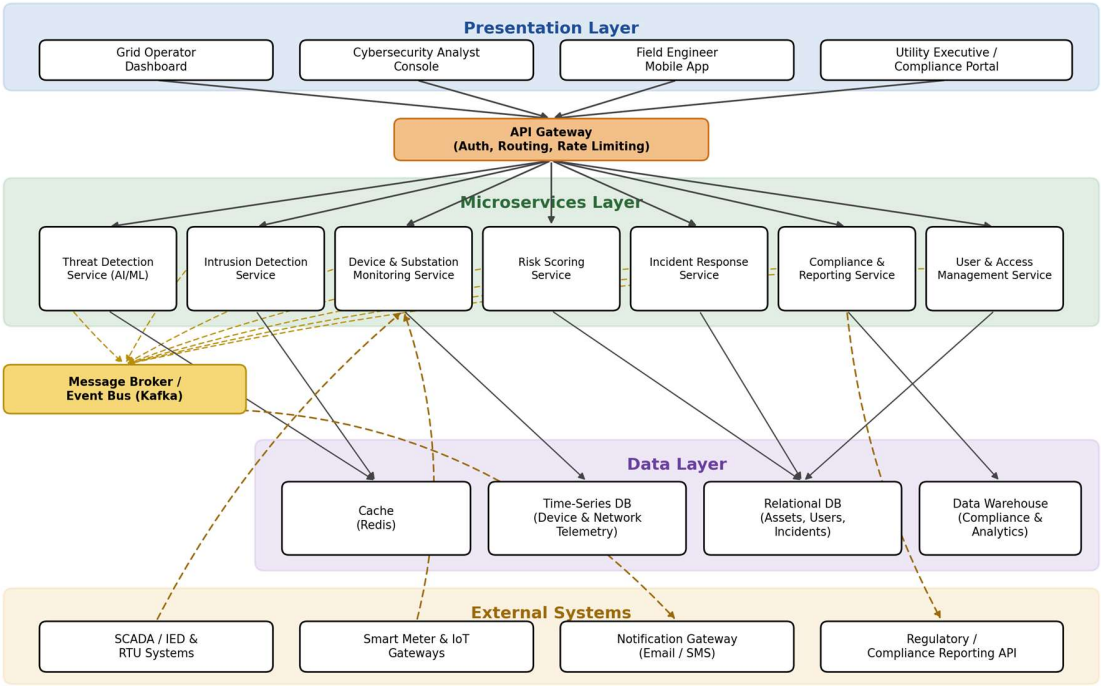


Figure 1: Proposed Microservices Architecture — Smart Grid Cybersecurity Management Platform

4. Explain Components and Their Interactions

4.1 Component Responsibilities

Component	Responsibility
Grid Operator Dashboard / Analyst Console / Field App	Client interfaces used by operators, analysts, field engineers, and executives to interact with the platform.
API Gateway	Single entry point that authenticates requests, applies role-based access control, routes traffic to the correct microservice, and enforces rate limiting.
Threat Detection Service (AI/ML)	AI/ML engine that analyzes network and device telemetry to detect known and unknown cyber threats in real time.

Component	Responsibility
Intrusion Detection Service	Correlates network flow data with device-level telemetry to detect lateral movement and unauthorized access across IT and OT segments.
Device & Substation Monitoring Service	Maintains a real-time status and communication-health inventory of substations, smart meters, and field devices.
Risk Scoring Service	Quantifies severity and likelihood of detected events and prioritizes them for response.
Incident Response Service	Executes or authorizes containment actions and tracks incident resolution status.
Compliance & Reporting Service	Aggregates data from all services into consolidated KPIs, executive dashboards, and scheduled/on-demand compliance reports.
User & Access Management Service	Authenticates users, enforces role-based permissions, and maintains an audit trail of key actions.
Message Broker / Event Bus	Decouples event producers (field devices, meters) from consumers (detection, monitoring, notifications), enabling asynchronous, scalable data flow.
Data Layer (Relational DB, Time-Series DB, Cache, Data Warehouse)	Persists transactional data, high-volume telemetry, frequently accessed data, and historical data for compliance analytics, respectively.
External Systems (SCADA/IED, Smart Meters, Notification Gateway, Regulatory API)	Third-party systems integrated via secure REST/EDI APIs for device telemetry, shipment-equivalent status, and email/SMS alerts.

4.2 Communication and Workflow

Clients communicate with the platform exclusively through the API Gateway, which authenticates each request and routes it to the relevant microservice over secure REST/HTTPS APIs. Synchronous, request-response interactions — such as an analyst requesting a device status or an executive opening a dashboard — flow through the gateway directly to the corresponding service, which reads from its own data store (using a cache for frequently requested data such as recent risk scores).

High-frequency, continuous data — substation telemetry, smart meter readings, and inter-service notifications — flow through the message broker instead of direct synchronous calls. For example, when a substation reports an anomalous reading, the Device Monitoring Service publishes an event to the broker; the Threat Detection and Notification services subscribe to this event independently, updating field engineers and the executive dashboard without the Device Monitoring Service needing to know about either downstream consumer. Similarly, the Intrusion Detection Service consumes network events published to the broker, and once it detects a significant anomaly, it publishes a threat alert that the Risk Scoring and Reporting services can subscribe to.

The overall workflow proceeds as follows: (1) device and network telemetry feeds the Threat Detection and Intrusion Detection services; (2) confirmed anomalies are scored by the Risk Scoring Service; (3) high-risk events trigger response actions through the Incident Response Service; (4) affected devices are tracked by the Device & Substation Monitoring Service until resolution; and (5) the Compliance & Reporting Service continuously aggregates outputs from every stage into consolidated dashboards and reports for stakeholders.

This event-driven workflow ensures that no single service becomes a bottleneck: the Threat Detection Service can process a burst of telemetry without waiting for the Incident Response Service to finish handling a prior alert, and the Compliance & Reporting Service can build its dashboards from the same event stream without placing additional query load on the operational services. This separation of concerns is what allows the platform to keep detection, response, and reporting responsive even during periods of unusually high grid activity.

5. Discuss Quality Attributes

5.1 Scalability

Each microservice can be scaled independently based on its own load profile. During attack surges, the Threat Detection Service can be scaled out on additional compute instances, while lighter services such as User & Access Management remain at baseline capacity. The event-driven backbone further allows telemetry volume to grow without requiring synchronous capacity across every consumer.

5.2 Security

The API Gateway centralizes authentication and enforces role-based access control with multi-factor authentication for administrative and field-facing roles, so individual services do not need to duplicate security logic. Data is encrypted in transit (HTTPS/TLS) and at rest across all data stores, and the User & Access Management Service maintains a full audit trail of sensitive actions such as containment approvals.

5.3 Performance

Separating the compute-intensive detection workload from transactional services prevents threat analysis from degrading dashboard or reporting response times. A caching layer serves frequently requested risk scores and device status, helping meet the 2-second alert and 3-second dashboard load targets, while the message broker ensures telemetry updates propagate to consumers within the required 15-minute window.

5.4 Reliability

Because services are loosely coupled through the message broker, a temporary failure or slowdown in one service — for example, the Device Monitoring Service during a field-gateway outage — does not

cascade to Threat Detection, Risk Scoring, or Reporting services. Failed events can be retried or queued in the broker without permanent data loss, improving overall system resilience.

5.5 Maintainability

Each microservice has clear, bounded responsibilities, so the AI/ML detection model can be retrained and redeployed independently of the rest of the platform. New detection algorithms, additional device integrations, or new report types can be introduced as isolated changes without requiring a full-system redeployment.

5.6 Availability

Multiple service instances run behind the API Gateway with health-checked load balancing, and the data layer uses replication for the relational database and data warehouse. This redundancy, combined with fault isolation between services, supports the target of 99.9% platform uptime.

6. Justify Architectural Decisions

6.1 Rationale

The combination of microservices, an API Gateway, and an event-driven message broker was chosen because the platform's workloads are fundamentally heterogeneous: compute-intensive AI/ML detection, transactional incident and compliance operations, and high-frequency streaming telemetry from substations and meters each have different scaling, latency, and reliability needs. A single unified style — whether a monolith or a simple layered design — would force all of these workloads to share the same deployment and scaling unit, making it difficult to meet the scalability and availability requirements identified in Section 1.3.

6.2 Trade-offs Considered

- **Operational complexity:** Microservices introduce additional operational overhead (service discovery, distributed monitoring, inter-service network calls) compared to a monolith. This is accepted because the independent scalability and fault isolation benefits outweigh the added complexity for critical infrastructure of this scale.
- **Eventual consistency:** Using an event-driven broker for cross-service updates (e.g., device status reaching the dashboard) means some data is only eventually consistent rather than instantly consistent. This is acceptable given the 15-minute telemetry requirement, and is far preferable to tightly coupling every service through synchronous calls.
- **Infrastructure cost:** Running multiple independently deployed services and a message broker has a higher baseline infrastructure cost than a monolith. This is justified by the reduced business cost of undetected attacks, downtime, and slower incident response.

6.3 Support for Future Enhancements and Long-Term Maintainability

The chosen architecture supports future growth in several ways. New capabilities — such as an offline/degraded mode for substations with intermittent connectivity, explainable-AI features for threat predictions, or additional regional grid integrations — can be added as new services or extensions to

existing ones without redesigning the platform. The API Gateway and event-driven broker make it straightforward to onboard additional integrations (new device vendors, new regulatory systems) by adding routing rules and event subscribers rather than modifying core business logic. Because each service is independently deployable and testable, the platform can evolve incrementally, reducing regression risk and supporting long-term maintainability as the utility's grid infrastructure grows.

7. Conclusion

This Architecture Design Analysis has examined the Smart Grid Cybersecurity Management Platform's system requirements and quality attributes, selected and justified a cloud-native microservices architecture with event-driven communication, presented the overall architecture diagram, explained component responsibilities and interactions, evaluated the design against key quality attributes, and justified the architectural trade-offs and long-term maintainability — providing a sound architectural foundation for subsequent detailed design and implementation.

The resulting design directly addresses the scalability, performance, reliability, security, maintainability, and interoperability requirements identified during requirements analysis, while remaining flexible enough to absorb future growth in device count, regional coverage, and regulatory scope. Subsequent phases of the project can build on this architectural foundation to produce detailed interface specifications, deployment topologies, and test plans for each microservice without revisiting the core structural decisions made here.